

Git & GitHub Basics

What is VCS (Version Control System)?

A **Version Control System (VCS)** is a tool that helps manage and track changes to files—especially source code—over time. It enables **collaboration**, **version history**, and **change management**, which are crucial for software development and project coordination.

Popular VCS Tools

<u>Tool</u>	<u>Type</u>	<u>Description</u>
Git	DVCS	Most popular distributed system; supports branching, merging, and offline workflows.
Subversion (SVN)	CVCS	Centralized version control, often used in enterprise environments.
Mercurial	DVCS	Simple and user-friendly alternative to Git.
Perforce	CVCS	Enterprise-grade tool for large-scale projects.

DVCS (Distributed VCS): Each user has a full local copy of the repository.

CVCS (Centralized VCS): There's one central repository; users commit directly to it.

Why Do We Need VCS?

1. **Tracking Changes:** Keeps record of every modification with timestamps and authors.
2. **Collaboration:** Multiple developers can work simultaneously without overwriting work.
3. **Branching & Merging:** Enables safe experimentation and integration of features.
4. **Version History:** Easy rollback to older versions.
5. **Conflict Resolution:** Helps manage and resolve code conflicts.
6. **Backup & Recovery:** Acts as a backup mechanism for your codebase.

Introduction to Git

Git is a distributed version control system developed by *Linus Torvalds* in 2005.

It is fast, flexible, and supports non-linear development through branches and merges.

Git is the engine — GitHub, GitLab, and Bitbucket are hosting platforms built on top of it.

Git Cheat Sheet

Command	Description
<code>git init</code>	Initialize a new Git repository
<code>git clone <url></code>	Clone a repository from a remote URL
<code>git status</code>	Show the current state of working directory
<code>git add <file></code>	Stage specific file(s) for commit
<code>git add .</code>	Stage all modified files
<code>git commit -m "message"</code>	Commit staged changes with a message
<code>git push</code>	Push commits to a remote repository
<code>git pull</code>	Fetch and merge changes from a remote
<code>git branch</code>	List existing branches
<code>git branch <name></code>	Create a new branch
<code>git checkout <branch></code>	Switch to another branch
<code>git merge <branch></code>	Merge another branch into current
<code>git log</code>	View commit history
<code>git diff</code>	Show file differences
<code>git reset <file></code>	Unstage a file
<code>git stash</code>	Temporarily save uncommitted changes
<code>git stash pop</code>	Reapply stashed changes
<code>git remote add <name> <url></code>	Connect local repo to remote
<code>git fetch</code>	Download objects and references
<code>git rebase <branch></code>	Reapply commits onto another branch

Understanding the .git/ Directory

When you run `git init`, Git creates a hidden directory called `.git/`.

This directory contains all necessary metadata and version history for your repository.

`.git`

├─ config

├─ HEAD

├─ hooks/

| └─ prepare-commit-msg.sample

├─ objects/

| └─ info/

| └─ pack/

└─ refs/

├─ heads/

└─ tags/

Directory Breakdown

- `config`: Repository-specific configuration (user, remote info, etc.)
- `HEAD`: Points to the current branch reference.
- `hooks`: Contains scripts triggered by Git events (pre/post commit, push, etc.).
- `objects`: Stores the actual Git data — commits, trees, blobs.
- `refs`: Contains pointers to commits (branches, tags, etc.).

⚙️ The `.git/` folder is the heart of your repository — never delete or modify it manually.

Git on MacBook (macOS Notes)

Setup

1. Check Git installation
2. `git --version`

If not installed, macOS will prompt to install Xcode Command Line Tools.

3. Set your identity

4. `git config --global user.name "Your Name"`
5. `git config --global user.email "you@example.com"`
6. Generate SSH Key (for GitHub)
7. `ssh-keygen -t rsa -b 4096 -C "you@example.com"`

Add the SSH key to your GitHub account (via *Settings* → *SSH and GPG keys*).

8. Connect to Remote
9. `git remote add origin git@github.com:username/repo.git`
10. `git push -u origin main`
11. Show hidden files (like `.git/`) in Finder
Press:
12. Command + Shift + .

Toggles visibility of hidden files and folders.

Difference – Git and Github

In Simple Terms:

Git = The tool you use on your computer to track changes in code.

GitHub = The online platform where you store and share that code with others.

Analogy:

Think of Git as Microsoft Word that saves versions of your document.

And GitHub as Google Drive where you upload that document so your team can view, comment, or edit together.

Quick Summary

Concept	Description
VCS	Tracks and manages code changes
Git	Distributed version control system
GitHub	Cloud platform for hosting and collaboration
.git/	Hidden folder that stores all repository data
Mac Tip	Use <code>Cmd + Shift + .</code> to view hidden .git

